The scikit-learn API

Estimators, transformers, predictors – and five design principles

Marcus Birkenkrahe

COSC 3430 · Artificial Intelligence · Augustana University

Based on Géron, *Hands-On Machine Learning with Scikit-Learn and PyTorch*, Ch. 2, and Buitinck et al. (2013)

Fall 2026

Colab starter notebook: https://birkenkrahe.github.io/go/sklearn

Roadmap

Ten steps, over several code-along lectures:

1. Look at the big picture
2. Get the data
3. Explore and visualize the data
4. Create the test data
5. Prepare the data for learning ✓
6. Select a model and train it
7. Fine-tune the model
8. Present your solution
9. Launch the production system
10. Monitor and maintain it

Today: before step 6, a look at the tools – how scikit-learn is designed, and why all its classes work alike.

Review

You used `SimpleImputer`, `OneHotEncoder`, `StandardScaler`, `Pipeline`, `ColumnTransformer`, `KNeighborsRegressor`, `LinearRegression`.

Which method did you call on every one of them?

`fit()` – or `fit_transform()`, which calls `fit()` first.
Everything a scikit-learn object learns from data, it learns in `fit()`.

What is scikit-learn?

- A Python **library**: classes and functions you import into your own programs – no graphical user interface, you use it from a script or a notebook
- Built on **NumPy** (fast numeric arrays) and **SciPy** (numerical routines, many of them wrappers around C, C++ and Fortran code)
- Started in 2007, first public release in 2010 (Pedregosa et al., 2011): well-established ML methods for people who are not ML experts
- The design of its interface (API) is described in Buitinck et al. (2013) – Géron summarizes it in Chapter 2

Three roles

Every scikit-learn object plays one or more of three roles:

Role	Method	Example
estimator	`fit()`	`SimpleImputer`
transformer	`transform()`, `fit_transform()`	`StandardScaler`
predictor	`predict()`, `score()`	`LinearRegression`

Every transformer and every predictor is also an estimator: it has to `fit()` before it can transform or predict.

An estimator: hyperparameters and learned parameters

→ **Switch to editor · code-along 1 (demo)**

```python
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")
print(imputer.strategy)
print(hasattr(imputer, "statistics_"))
imputer.fit(housing_num)
print(hasattr(imputer, "statistics_"))
print(imputer.statistics_)
print(housing_num.median().values)
```

```
median
False
True
[-118.51   34.26   29.    2125.
  434.   1167.   408.    3.5385]
[-118.51   34.26   29.    2125.
  434.   1167.   408.    3.5385]
```

- **Hyperparameter**: set in the constructor, no underscore (`strategy`)
- **Learned parameter**: set by `fit()`, ends with an underscore (`statistics_`) – not there before `fit()`

A transformer: fit on the training set only

→ **Switch to editor · code-along 2 (demo)**

```python
from sklearn.preprocessing import StandardScaler

X_train = housing_num[["median_income"]]
X_test = strat_test_set[["median_income"]]

scaler = StandardScaler()
scaler.fit(X_train)
print(scaler.mean_, scaler.scale_)

X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
print(X_train_scaled.mean().round(3),
      X_train_scaled.std().round(3))
print(X_test_scaled.mean().round(3),
      X_test_scaled.std().round(3))
```

```
[3.87042814] [1.89187853]
0.0 1.0
0.001 1.021
```

`fit()` learns from the training set, `transform()` applies it to any data.

The test set is close to 0 and 1, not exactly: it was scaled with the **training** set's mean and standard deviation.

A predictor: `predict()` and `score()`

→ **Switch to editor · code-along 3 (demo)**

```
from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(X_train, housing_labels)
print(lin_reg.coef_, lin_reg.intercept_)
print(lin_reg.predict(X_train[:3]).round(-2))
print(housing_labels[:3].values)
print(round(lin_reg.score(X_train,
                          housing_labels), 2))
```

```
[41956.98204657] 43942.03453763187
[132000. 299400. 146000.]
[458300. 483800. 101700.]
0.47
```

`score()` of a regressor is $R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$: 1 is a perfect fit, 0 is no better than always predicting the mean $\bar{y}$.

Income alone explains about half of the variation – the first district is off by more than $300,000. Better models: next step.

Which roles does each class play?

	estimator	**transformer**	**predictor**
`SimpleImputer`, `OneHotEncoder`, `StandardScaler`	✓	✓	
`LinearRegression`, `KNeighborsRegressor`	✓		✓
`KMeans`	✓	✓	✓
`Pipeline`	✓	last step	last step

- `KMeans`: `fit()` finds the centers, `transform()` returns the distance to each center, `predict()` the nearest center
- A `Pipeline` plays the role of its last step
- Check it yourself: `hasattr(est, "transform")`, `hasattr(est, "predict")`

Five design principles

1. **Consistency**: all objects share one simple interface – the three roles
2. **Inspection**: hyperparameters and learned parameters are public attributes
3. **Non-proliferation of classes**: data sets are NumPy arrays (or SciPy sparse matrices), hyperparameters plain strings and numbers
4. **Composition**: existing building blocks are reused, e.g. in a `Pipeline`
5. **Sensible defaults**: every hyperparameter has a reasonable default value

Inspection

→ **Switch to editor · code-along 4 (demo)**

```
print(imputer.get_params())
print([name for name in vars(imputer) if name.endswith("_")])
```

```
{'add_indicator': False, 'copy': True, 'fill_value': None,
 'keep_empty_features': False, 'missing_values': nan, 'strategy': 'median'}
['feature_names_in_', 'n_features_in_', 'indicator_', 'statistics_']
```

- `get_params()`: all hyperparameters; `vars()`: everything the object stores
- `fit()` also records which columns it saw (`feature_names_in_`), so `transform()` can check that it gets the same columns later

Sensible defaults

→ **Switch to editor · code-along 5 (demo)**

```
from sklearn.neighbors import KNeighborsRegressor

print(KNeighborsRegressor().get_params())
print(SimpleImputer().get_params()["strategy"])
```

```
{'algorithm': 'auto', 'leaf_size': 30, 'metric': 'minkowski',
 'metric_params': None, 'n_jobs': None, 'n_neighbors': 5, 'p': 2,
 'weights': 'uniform'}
mean
```

- kNN: 5 neighbors, equal weights, Minkowski metric with p=2 – the Euclidean distance
- `SimpleImputer`: the mean by default – we chose the median because of the heavy tails

Non-proliferation of classes

→ **Switch to editor · code-along 6 (demo)**

```
scaled = StandardScaler().fit_transform(housing_num)
print(type(scaled))

scaler_pd = StandardScaler().set_output(
    transform="pandas")
scaled_df = scaler_pd.fit_transform(housing_num)
print(type(scaled_df))
print(scaled_df[["total_rooms",
                 "median_income"]].head(3).round(2))
```

```
<class 'numpy.ndarray'>
<class 'pandas.core.frame.DataFrame'>
       total_rooms  median_income
13096         0.31          -0.94
14973        -0.31           1.17
3785         -0.71          -0.76
```

Transformers return NumPy arrays: no column names, no index – that is why we needed `get_loc` in part 6.

`set_output(transform="pandas")` (since version 1.2) returns a `DataFrame` instead.

Composition: a pipeline is a predictor

→ **Switch to editor · code-along 7**

```
knn_pipeline = make_pipeline(SimpleImputer(strategy="median"),
                             StandardScaler(),
                             KNeighborsRegressor())
print(knn_pipeline.get_params()[
    "kneighborsregressor__n_neighbors"])

X_test_num = strat_test_set[housing_num.columns]
y_test = strat_test_set["median_house_value"]
knn_pipeline.fit(housing_num, housing_labels)
print(round(knn_pipeline.score(X_test_num, y_test), 3))

knn_pipeline.set_params(kneighborsregressor__n_neighbors=10)
knn_pipeline.fit(housing_num, housing_labels)
print(knn_pipeline[-1])
print(round(knn_pipeline.score(X_test_num, y_test), 3))
```

```
5
0.701
KNeighborsRegressor(n_neighbors=10)
0.713
```

The pipeline has `fit()`, `predict()`, `score()` – because its last step does.

Step hyperparameters: `step__name` (two underscores). Fine-tuning will use `set_params()`.

Consistency pays off

Every predictor has `fit()`, `predict()` and `score()`.

Swap `KNeighborsRegressor()` for `LinearRegression()` in the pipeline – nothing else changes.

Next step: try several models with the same preparation, compare their scores, keep the best.

Practice: a class you have not used

Notebook section **Practice**: `RobustScaler` (in `sklearn.preprocessing`) is new to you. Find out with the API alone:

1. Which roles does it play?
   Tip: `hasattr(robust, "transform")`, same for `"fit"`, `"predict"`
2. What are its hyperparameters, and their defaults?
   Tip: `robust.get_params()`
3. Fit it on `housing_num[["population"]]`. What did it learn?
   Tip: `vars(robust)` – learned attributes end with `_`
4. Can you compute the same values with pandas?
   Tip: `housing_num["population"].describe()`

Roles	estimator, transformer
Defaults	`quantile_range=(25.0, 75.0)`, `with_centering=True`, ...
Learned	`center_` = **1167**, `scale_` = **939**
pandas	median (50%), and 75% minus 25% (1726 − 787)

Median and interquartile range hardly move when a few districts have a huge population: the heavy tail does not distort the scaling.

What we learned

- **Three roles**: estimator (`fit()`), transformer (`transform()`, `fit_transform()`), predictor (`predict()`, `score()`) – one object can play several
- **Hyperparameters** are set in the constructor; **learned parameters** end with _ and exist after `fit()`
- **Five principles**: consistency, inspection, non-proliferation of classes, composition, sensible defaults
- **Checkout**: you get a scikit-learn class you have never seen. Which three things can you find out about it without its documentation – and how?

Next: select a model and train it.